

# MI GFX API

---

## 1 Overview

---

### 1.1. Module Description

GFX (Graphic Engine) hardware provides fast graphics rendering for user-painted UI.

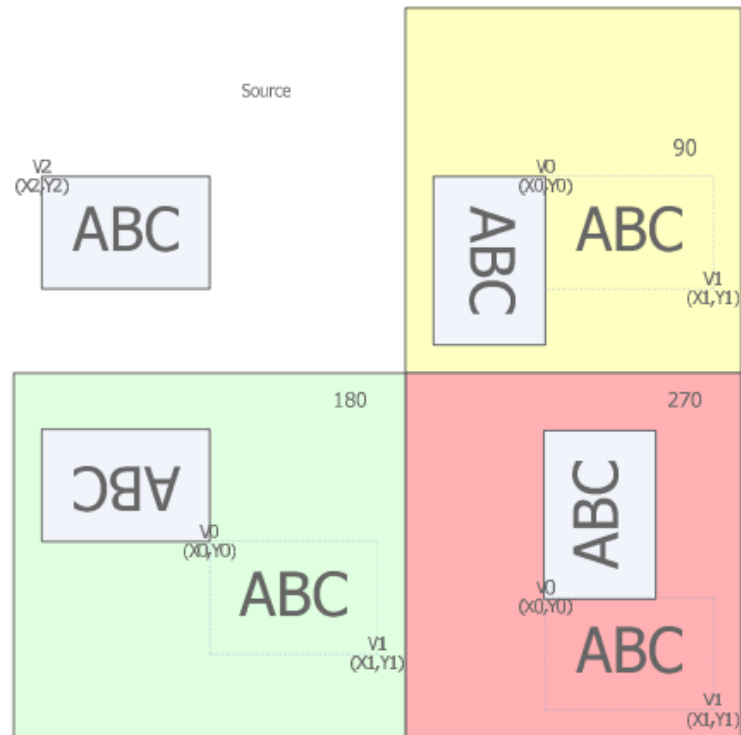
There are:

- rectangle color fill
- Bitmap moving (support scaling, rotation, mirror flip, format conversion, alpha blending overlay, ColorKey, etc.)

Currently only Tauyaki, Takoyaki and Ikayaki series chips support GFX.

**Rotate: HW supports 90/180/270 degree rotation**

## Rotate



Picture 1-1

**Alpha Blend Overlay: supports normal 2D alpha manipulation**

## Alpha Blending

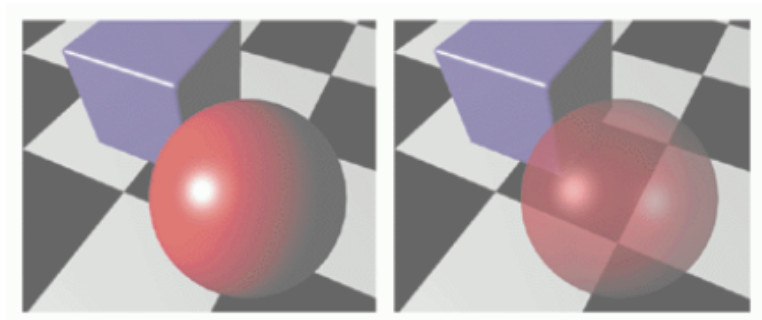


Figure 1-2

**Mirroring: Support H mirroring, V mirroring, HV mirroring at the same time**

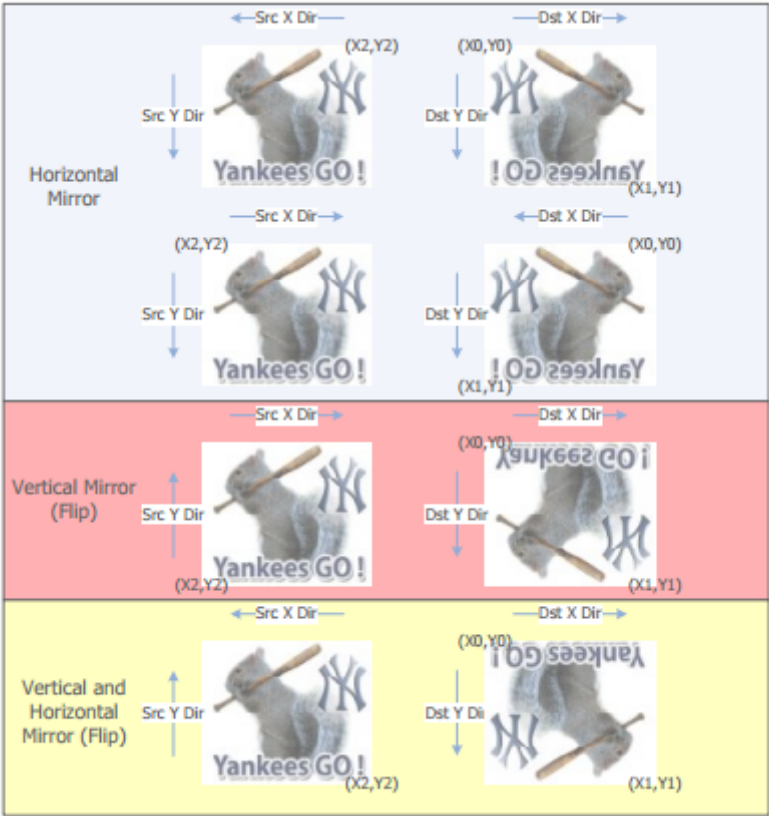


Figure 1-3

Scaling: support scaling during bitblt

Stretch Bitblt

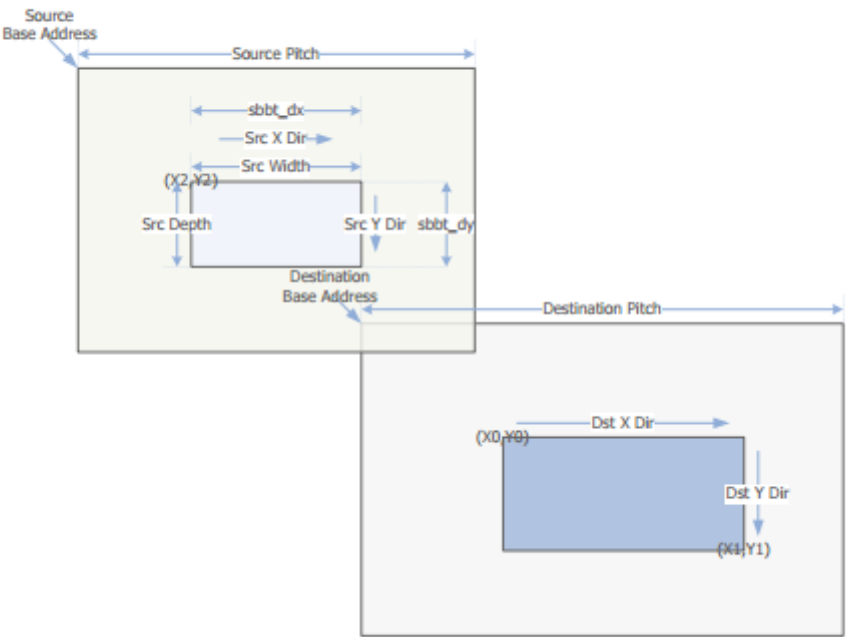


Figure 1-4

Other operations are not listed one by one, which is basically the same as the standard 2D acceleration.

---

1.2. Call flow diagram

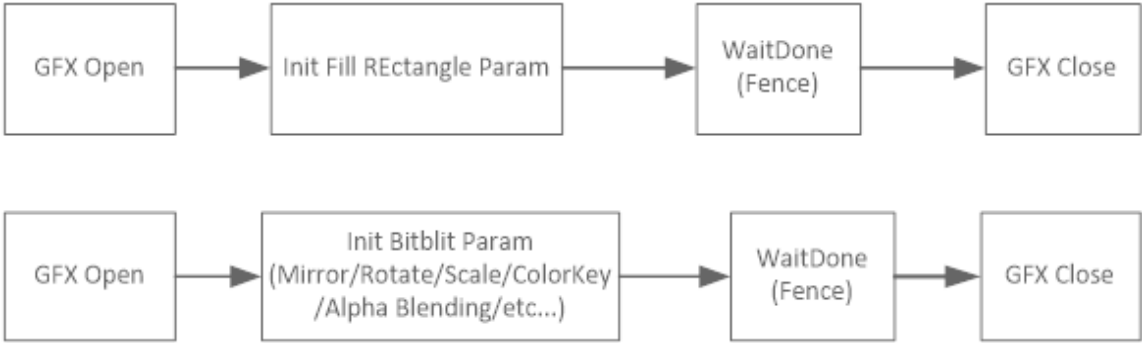


Figure 1-5

If users want to use GE to process video or UI data, they can use the following process

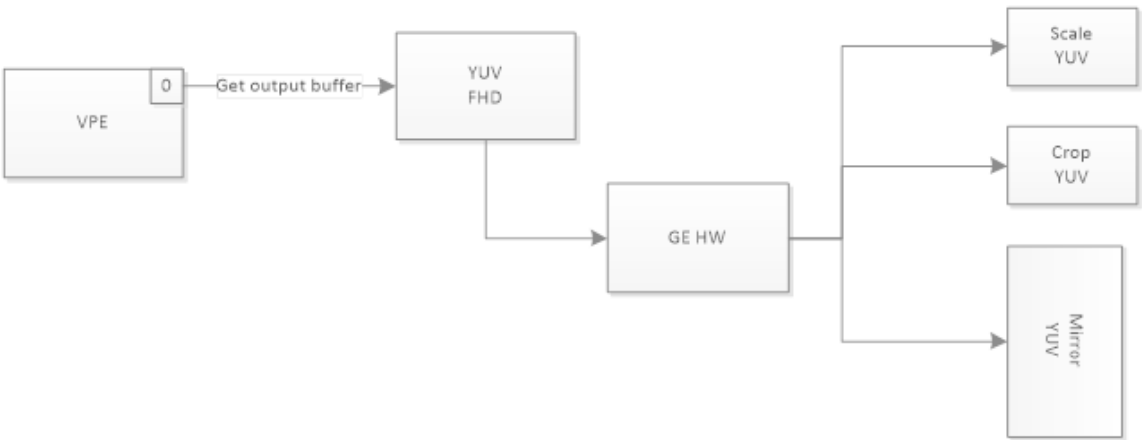


Figure 1-6

1.3. Keyword Description

- Fence

Corresponding to the state that the operation of a GFX needs to wait, it will block and wait, but there will also be a timeout mechanism inside to prevent

fools.

- Colorkey

Key color, for example, when moving, you can remove a certain color in the bitmap, which can set this color as the key color

- Pitch

The number of bytes occupied by each line of pixels,  $(\text{pixel bits}/8) * \text{Width}$ , also corresponds to the commonly used Stride, LineLength

---

## 2. API Reference

---

### 2.1. Function Module API

| API名                                                                              | Function                                                                            |
|-----------------------------------------------------------------------------------|-------------------------------------------------------------------------------------|
| <a href="#">MI_GFX_Open</a> [#22-mi_gfx_open]                                     | Open the GFX device                                                                 |
| <a href="#">MI_GFX_Close</a> [#23-mi_gfx_close]                                   | Close the GFX device                                                                |
| <a href="#">MI_GFX_WaitAllDone</a> [#24-mi_gfx_waitalldone]                       | Wait for all tasks of GFX to complete                                               |
| <a href="#">MI_GFX_QuickFill</a> [#25-mi_gfx_quickfill]                           | Add a quick fill action to a task                                                   |
| <a href="#">MI_GFX_GetAlphaThresholdValue</a> [#26-mi_gfx_getalphathresholdvalue] | Get the alpha decision threshold                                                    |
| <a href="#">MI_GFX_SetAlphaThresholdValue</a> [#27-mi_gfx_setalphathresholdvalue] | Set alpha decision threshold                                                        |
| <a href="#">MI_GFX_Bitblit</a> [#28-mi_gfx_bitblit]                               | Add a move operation with additional functionality to the raster bitmap to the task |
| <a href="#">MI_GFX_SetPalette</a> [#29-mi_gfx_setpalette]                         | Set the palette for indexed color format                                            |
| <a href="#">MI_GFX_InitDev</a> [#210-mi_gfx_initdev]                              | Initialize GFX device                                                               |
| <a href="#">MI_GFX_DeInitDev</a> [#211-mi_gfx_deinitdev]                          | Deinitialize GFX device                                                             |
| <a href="#">MI_GFX_DrawLine</a> [#212-mi_gfx_drawline]                            | Add line drawing actions to tasks                                                   |

## 2.2. MI\_GFX\_Open

- Function

Call this interface to open the GFX device and initialize the hardware device.

- grammar

```
MI_ S32 MI_GFX_Open(void);
```

- return value
  - MI\_SUCCESS Open the device successfully.
  - Initialization failed, please refer to the [error code](#) [#jump] for details .
- rely
  - Header file: mi\_gfx.h
  - Library file: libmi\_gfx.a/libmi\_gfx.so
- Notice
  - This interface should be called first before GFX-related operations to ensure that the GFX device is turned on.
  - After the GFX device is initialized once, calling it again will return to initialized and can be called repeatedly.
- Example

```
1. MI_S32 s32Ret = 0;
2.
3. MI_SYS_Init();
4. /* open GFX device*/
5. s32Ret = MI_GFX_Open();
6. if (MI_SUCCESS != s32Ret)
7. {
8.     return -1;
9. }
10. /* close GFX device*/
11. MI_GFX_Close();
12. MI_SYS_Exit();
```

---

## 2.3. MI\_GFX\_Close

- Function

Call this interface to close the GFX device.
- grammar

```
MI_S32 MI_GFX_Close(void);
```

- return value
    - MI\_SUCCESS successfully destroyed the GFX module.
    - Destroying the GFX module failed, please refer to the [error code](#) [#jump] for details .
  - rely
    - Header file: mi\_gfx.h
    - Library file: libmi\_gfx.a/libmi\_gfx.so
  - Notice
    - The number of times to call [MI\\_GFX\\_Open](#) [#22-mi\_gfx\_open] and [MI\\_GFX\\_Close](#) [#23-mi\_gfx\_close] needs to correspond, and can be called repeatedly. When the number of times of Open is the same as the number of times of Close, the GFX api will no longer be called.
- 

## 2.4. MI\_GFX\_WaitAllDone

- Function

Call this interface to wait for all tasks of GFX to complete or wait for the specified task to complete.

- grammar

```
MI_S32 MI_GFX_WaitAllDone(MI_BOOL bWaitAllDone, MI_penU16 u16TargetFence);
```

- keepsake

| parameter name | describe                                                                        | input<br>Output |
|----------------|---------------------------------------------------------------------------------|-----------------|
| bWaitAllDone   | Whether to wait for all GFX tasks to finish                                     | enter           |
| u16TargetFence | Wait for the specified fence to complete, only valid when bWaitAllDone is FALSE | enter           |

- return value
  - MI\_SUCCESS All pending GFX tasks have completed.
  - Failed to wait for the GFX task to complete. For details, refer to the [error code](#) [#jump] .
- rely
  - Header file: mi\_gfx.h
  - Library file: libmi\_gfx.a/libmi\_gfx.so
- Notice
  - [Make sure to call MI\\_GFX\\_Open](#) [#22-mi\_gfx\_open] to open the GFX device before calling this interface .
  - This interface is a blocking interface and will block until all GFX tasks are completed.

---

## 2.5. MI\_GFX\_QuickFill

- Function

Fill in the color of u32ColorVal to the target area pstDstRect in the target bitmap pstDst to realize the function of color filling.

- grammar

```
MI_S32 MI_GFX_QuickFill(MI_GFX_Surface_t *pstDst, MI_GFX_Rect_t *pstDstRect, MI_U32 u32ColorVal, MI_U16 *pu16Fence);
```

- keepsake

| parameter name | describe                     | input Output |
|----------------|------------------------------|--------------|
| pstDst         | target bitmap                | enter        |
| pstDstRect     | target bitmap operation area | enter        |
| u32ColorVal    | fill color value             | enter        |
| pu16Fence      | the returned Fence pointer   | output       |

- return value
  - MI\_SUCCESS Successfully added the task of QuickFill behavior to the HW queue.
  - Filling the color block failed, please refer to the [error code](#) [#jump] for details .
- rely
  - Header files: mi\_gfx.h: mi\_gfx.h
  - Library file: libmi\_gfx.a/libmi\_gfx.so
- Notice
  - [Make sure to call MI\\_GFX\\_Open](#) [#22-mi\_gfx\_open] to open the GFX device before calling this interface .
  - The target bitmap operation area cannot exceed the size of the target bitmap.
  - If the pixel format of the target bitmap is RGB or ARGB, u32ColorVal is A8:R8:G8:B8 from high to low. If the pixel format of the target bitmap is YUYV422, then A8 is invalid, R corresponds to Y, and G corresponds to U, B correspond to V.
- Example

```

1. FILE *fp = NULL;
2. MI_PHY phyAddr;
3. void *pVirAddr;
4. MI_GFX_Surface_t stDst;
5. MI_GFX_Rect stDstRect;

```

```
6. MI_U32 u32ColorVal = 0xffffffff;
7. MI_U16 u16TargetFence;
8.
9. fp = fopen(SRC_FILE_NAME, "wb");
10.
11. ExecFunc(MI_SYS_Init(), MI_SUCCESS);
12. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0",
SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
13.
14. //MI_U32 phySrcAddr, phyDstAddr;
15. ExecFunc(MI_GFX_Open(), MI_SUCCESS);
16. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2,
&pVirAddr, FALSE), MI_SUCCESS);
17. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
18.
19. //fillrect
20. memset(&stDst, 0x0, sizeof(stDst));
21. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
22. stDst.u32Width = SRC_WIDTH;
23. stDst.u32Height = SRC_HEIGHT;
24. stDst.u32Stride = SRC_WIDTH * 2;
25. stDst.phyAddr = phyAddr;
26.
27. memset(&stDstRect, 0x0, sizeof(stDstRect));
28. stDstRect.s32Xpos = 100;
29. stDstRect.s32Ypos = 100;
30. stDstRect.u32Width = 100;
31. stDstRect.u32Height = 100;
32.
33. ExecFunc(MI_GFX_QuickFill(&stDst, &stDstRect, u32ColorVal,
&u16TargetFence), MI_SUCCESS);
34. ExecFunc(MI_GFX_WaitAllDone(FALSE, u16TargetFence),
MI_SUCCESS);
35. if (NULL != fp)
36. {
37.     fwrite(pVirAddr, 1, SRC_WIDTH*SRC_HEIGHT*2, fp);
38.     fclose(fp);
39.     fp = NULL;
40. }
41. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2),
MI_SUCCESS);
42. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
43.
44. ExecFunc(MI_GFX_Close(), MI_SUCCESS);
45. ExecFunc(MI_SYS_Exit(), MI_SUCCESS);
```

## 2.6. MI\_GFX\_GetAlphaThresholdValue

- Function

Get the alpha decision threshold.

When the source bitmap and the target bitmap do the bitblit operation, regardless of the pixel format of the source bitmap and the target bitmap, the hardware will each generate an intermediate bitmap whose pixel format is ARGB8888, and then output the operation result of the intermediate bitmap. to the target bitmap. When the pixel format of the target image is ARGB1555, if the alpha value of a pixel after the intermediate bitmap operation is less than this threshold, the alpha bit of the pixel output to the target bitmap will be 0; if it is greater than or equal to this threshold, the alpha bit of the pixel will be set to 0. 1.

- grammar

```
MI_S32 MI_GFX_GetAlphaThresholdValue(MI_U8 *pu8ThresholdValue);
```

- keepsake

| parameter name    | describe                                | input Output |
|-------------------|-----------------------------------------|--------------|
| pu8ThresholdValue | pointer to the alpha decision threshold | output       |

- return value

- MI\_SUCCESS successfully obtained the alpha threshold.
- Failed to get the alpha threshold. For details, see [Error Code](#) [#jump] .

- rely

- Header files: mi\_gfx.h: mi\_gfx.h
- Library file: libmi\_gfx.a/libmi\_gfx.so

## 2.7. MI\_GFX\_SetAlphaThresholdValue

- Function

Set the alpha decision threshold.

When the source bitmap and the target bitmap do the bitblit operation, regardless of the pixel format of the source bitmap and the target bitmap, the hardware will each generate an intermediate bitmap whose pixel format is ARGB8888, and then output the operation result of the intermediate bitmap. to the target bitmap. When the pixel format of the target image is ARGB1555, if the alpha value of a pixel after the intermediate bitmap operation is less than this threshold, the alpha bit of the pixel output to the target bitmap will be 0; if it is greater than or equal to this threshold, the alpha bit of the pixel will be set to 0. 1.

- grammar

```
MI_S32 MI_GFX_SetAlphaThresholdValue(MI_U8 u8ThresholdValue);
```

- keepsake

| number name      | describe                 | input Output |
|------------------|--------------------------|--------------|
| u8ThresholdValue | alpha decision threshold | enter        |

- return value
  - MI\_SUCCESS successfully sets the alpha threshold.
  - Failed to set alpha threshold. For details, see [Error Code](#) [#jump] .
- rely
  - Header files: mi\_gfx.h: mi\_gfx.h
  - Library file: libmi\_gfx.a/libmi\_gfx.so

2.8. MI\_GFX\_BitBlit

- Function

Operates the source bitmap (pstSrc) and the specified area (pstSrcRect, pstDstRect) of the target bitmap (pstDst), and copies the calculated bitmap

to the specified area (pstDstRect) of the target bitmap (pstDst).

- grammar

```
MI_S32 MI_GFX_BitBlit(MI_GFX_Surface_t *pstSrc, MI_GFX_Rect_t
*pstSrcRect, MI_GFX_Surface_t *pstDst, MI_GFX_Rect_t
*pstDstRect, MI_GFX_Opt_t *pstOpt,
MI_U16 *pu16Fence);
```

- keepsake

| parameter name | describe                               | input Output |
|----------------|----------------------------------------|--------------|
| pstSrc         | source bitmap.                         | enter        |
| pstSrcRect     | Source bitmap manipulation area.       | enter        |
| pstDst         | target bitmap.                         | enter        |
| pstDstRect     | The target bitmap manipulation area.   | enter        |
| pstOpt         | Operation parameter setting structure. | enter        |
| pu16Fence      | the returned Fence pointer             | output       |

- return value

- MI\_SUCCESS initialization succeeded.
- Return failed, please refer to the [error code](#) [#jump] for details .

- rely

- Header files: mi\_gfx.h: mi\_gfx.h
- Library file: libmi\_gfx.a/libmi\_gfx.so

- Notice

- [Make sure to call MI\\_GFX\\_Open](#) [#22-mi\_gfx\_open] to open the GFX device before calling this interface .
- When the size of the source bitmap operation area and the target bitmap operation area are inconsistent, the source area is scaled

according to the proportion of the target area, and then the operation is performed with the target area.

- During clip operation, the clipping area must have a common intersection with the target bitmap operation area, otherwise an error will be returned.
- The MI\_GFX\_Opt\_t structure stores the configuration information of the GFX operation function, such as: whether to use the color key (colorkey) and the configuration value of the colorkey; whether to perform the area clipping (clip operation) and specify the clip area; whether to mirror, whether to perform alpha blending and other information. The above operations can be enabled at the same time.
- Example

```

1. FILE *fp = NULL;
2. FILE *dstfp = NULL;
3. MI_PHY phyAddr, phyAddr2;
4. void *pVirAddr = NULL, *pVirAddr2 = NULL;
5. MI_GFX_Surface_t stSrc, stDst;
6. MI_GFX_Rect_t stSrcRect, stDstRect;
7. MI_U16 u16TargetFence;
8.
9. fp = fopen(SRC_FILE_NAME, "wb");
10. dstfp = fopen(DST_FILE_NAME, "wb");
11.
12. ExecFunc(MI_SYS_Init(), MI_SUCCESS);
13. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0",
SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
14. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0",
DST_WIDTH*360*2, &phyAddr2), MI_SUCCESS);
15.
16. //MI_U32 phySrcAddr, phyDstAddr;
17. ExecFunc(MI_GFX_Open(), MI_SUCCESS);
18. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2,
&pVirAddr, FALSE), MI_SUCCESS);
19. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
20.
21. ExecFunc(MI_SYS_Mmap(phyAddr2, DST_WIDTH*DST_HEIGHT*2,
&pVirAddr2, FALSE), MI_SUCCESS);
22. memset(pVirAddr2, 0x0F, DST_WIDTH*DST_HEIGHT*2);
23.
24. //bitblit
25. memset(&stSrc, 0x0, sizeof(stSrc));
26. stSrc.eColorFmt = E_MI_GFX_FMT_ARGB1555;

```

```
27. stSrc.u32Width = SRC_WIDTH;
28. stSrc.u32Height = SRC_HEIGHT;
29. stSrc.u32Stride = SRC_WIDTH * 2;
30. stSrc.phyAddr = phyAddr;
31.
32. memset(&stSrcRect, 0x0, sizeof(stSrcRect));
33. stSrcRect.s32Xpos = 100;
34. stSrcRect.s32Ypos = 100;
35. stSrcRect.u32Width = 300;
36. stSrcRect.u32Height = 300;
37.
38. memset(&stDst, 0x0, sizeof(stDst));
39. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
40. stDst.u32Width = DST_WIDTH;
41. stDst.u32Height = DST_HEIGHT;
42. stDst.u32Stride = DST_WIDTH * 2;
43. stDst.phyAddr = phyAddr2;
44.
45. memset(&stDstRect, 0x0, sizeof(stDstRect));
46. stDstRect.s32Xpos = 200;
47. stDstRect.s32Ypos = 100;
48. stDstRect.u32Width = 200;
49. stDstRect.u32Height = 100;
50.
51. ExecFunc(MI_GFX_BitBlit(&stSrc, &stSrcRect, &stDst,
&stDstRect, NULL, &u16TargetFence), MI_SUCCESS);
52. ExecFunc(MI_GFX_WaitAllDone(FALSE, u16TargetFence),
MI_SUCCESS);
53. if (NULL != dstfp)
54. {
55.     fwrite(pVirAddr2, 1, DST_WIDTH*DST_HEIGHT*2, dstfp);
56.     fclose(dstfp);
57.     dstfp = NULL;
58. }
59.
60. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2),
MI_SUCCESS);
61. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
62.
63. ExecFunc(MI_SYS_Munmap(pVirAddr2, DST_WIDTH*DST_HEIGHT*2),
MI_SUCCESS);
64. ExecFunc(MI_SYS_MMA_Free(phyAddr2), MI_SUCCESS);
65.
66. ExecFunc(MI_GFX_Close(), MI_SUCCESS);
67. ExecFunc(MI_SYS_Exit(), MI_SUCCESS);
```

## 2.9. MI\_GFX\_SetPalette

- Function  
Set palette for indexed color format (I2/I4/I8)

- grammar

```
MI_S32 MI_GFX_SetPalette(MI_GFX_ColorFmt_e eColorFmt,
MI_GFX_Palette_t *pstPalette);
```

- keepsake

| parameter name | describe                                      | input Output |
|----------------|-----------------------------------------------|--------------|
| eColorFmt      | The color format corresponding to the palette | enter        |
| pstPalette     | palette data                                  | enter        |

- return value
  - MI\_SUCCESS initialization succeeded.
  - Return failed, please refer to the [error code](#) [#jump] for details .
- rely
  - Header files: mi\_gfx.h: mi\_gfx.h
  - Library file: libmi\_gfx.a/libmi\_gfx.so
- Notice
  - [Make sure to call MI\\_GFX\\_Open](#) [#22-mi\_gfx\_open] to open the GFX device before calling this interface .
  - [pstPalette is a MI\\_GFX\\_PaletteEntry\\_t](#) [#313-mi\_gfx\_paletteentry\_t] array with a capacity of 256 , each [MI\\_GFX\\_PaletteEntry\\_t](#) [#313-mi\_gfx\_paletteentry\_t] represents a color, and the corresponding subscript in the array represents the color index.
- Example

```
1. MI_GFX_Palette_t myPal;
2. MI_GFX_PaletteEntry_t index_0;
```

```

3. MI_GFX_PaletteEntry_t index_1;
4. index_0.RGB.u8A = 0x40;
5. index_0.RGB.u8R = 0xFF;
6. index_0.RGB.u8G = 0;
7. index_0.RGB.u8B = 0;
8. index_1.RGB.u8A = 0x40;
9. index_1.RGB.u8R = 0;
10. index_1.RGB.u8G = 0xFF;
11. index_1.RGB.u8B = 0;
12.
13. myPal.aunPalette[0] = index_0;
14. myPal.aunPalette[1] = index_1;
15. myPal.u16PalStart = 0
16. myPal.u16PalEnd = 1;
17. MI_GFX_SetPalette(E_MI_GFX_FMT_I8, &myPal);

```

## 2.10. MI\_GFX\_InitDev

- Function

Initialize the gfx device.

- grammar

```
MI_S32 MI_GFX_InitDev(MI_GFX_InitParam_t *pstInitParam);
```

- keepsake

| parameter name | describe                         | input Output |
|----------------|----------------------------------|--------------|
| pstInitParam   | Device initialization parameters | enter        |

- return value
  - MI\_SUCCESS succeeded.
  - Failed, please refer to the error code for details.
- rely
  - Header file: mi\_gfx.h
  - Library file: libmi\_gfx.a/libmi\_gfx.so

---

## 2.11. MI\_GFX\_DeInitDev

- Function

Anti-initial gfx device.

- grammar

```
MI_ S32 MI_GFX_DeInitDev (void);
```

- return value
  - MI\_SUCCESS succeeded.
  - Failed, please refer to the error code for details.
- rely
  - Header file: mi\_gfx.h
  - Library file: libmi\_gfx.a/libmi\_gfx.so
- Notice
  - This function must be called after initializing the device, otherwise it returns failure.
  - If this interface is not called before the app exits, the device will be automatically deinitialized internally.
- Example

```
1. MI_GFX_Palette_t myPal;  
2. MI_GFX_PaletteEntry_t index_0;  
3. MI_GFX_PaletteEntry_t index_1;  
4. index_0.RGB.u8A = 0x40;  
5. index_0.RGB.u8R = 0xFF;  
6. index_0.RGB.u8G = 0;  
7. index_0.RGB.u8B = 0;  
8. index_1.RGB.u8A = 0x40;
```

```
9. index_1.RGB.u8R = 0;

10. index_1.RGB.u8G = 0xFF;

11. index_1.RGB.u8B = 0;

12. myPal.aunPalette[0] = index_0;

13. myPal.aunPalette[1] = index_1;

14. myPal.u16PalStart = 0

15. myPal.u16PalEnd = 1;

16. MI_GFX_SetPalette(E_MI_GFX_FMT_I8,&myPal);
```

2.12. MI\_GFX\_DrawLine

- Function  
Add a line drawing operation to the task.
- grammar

```
MI_S32 MI_GFX_DrawLine(MI_GFX_Surface_t *pstDst, MI_GFX_Line_t
*pstLine, MI_U16 *pu16Fence);
```

- keepsake

| parameter name | describe                       | input Output |
|----------------|--------------------------------|--------------|
| pstDst         | target bitmap                  | enter        |
| pstLine        | Line properties                | enter        |
| pu16Fence      | Returns a pointer to the Fence | output       |

- return value
  - MI\_SUCCESS succeeded.

- Failed, please refer to the error code for details.
- rely
  - Header file: mi\_gfx.h
  - Library file: libmi\_gfx.a/libmi\_gfx.so
- Notice
  - Make sure to call MI\_GFX\_Open to open the GFX device before calling this interface.
  - The drawing area cannot exceed the size of the target bitmap.
  - The end point of the line will not be drawn.
  - If the pixel format of the target bitmap is RGB or ARGB, u32ColorFrom/u32ColorFrom is A8:R8:G8:B8 from high to low, if the pixel format of the target bitmap is YUYV422, then A8 is invalid, R corresponds to Y, G corresponds to U, and B corresponds to V.
- Example

```

1. FILE *fp = NULL;
2. MI_GFX_Surface_t stDst;
3. MI_GFX_Line_t stLine;
4. MI_U16 u16TargetFence;
5. MI_PHY phyAddr;
6. void *VirAddr;
7.
8. ExecFunc(MI_SYS_Init(), MI_SUCCESS);
9. fp = fopen(SRC_FILE_NAME, "wb");
10. ExecFunc(MI_SYS_MMA_Alloc("mma_heap_name0",
SRC_WIDTH*SRC_HEIGHT*2, &phyAddr), MI_SUCCESS);
11.
12. //MI_U32 phySrcAddr, phyDstAddr;
13. ExecFunc(MI_GFX_Open(), MI_SUCCESS);
14. ExecFunc(MI_SYS_Mmap(phyAddr, SRC_WIDTH*SRC_HEIGHT*2,
&pVirAddr, FALSE), MI_SUCCESS);
15. memset(pVirAddr, 0x22, SRC_WIDTH*SRC_HEIGHT*2);
16.
17. //draw line
18. memset(&stDst, 0x0, sizeof(stDst));
19. stDst.eColorFmt = E_MI_GFX_FMT_ARGB1555;
20. stDst.u32Width = SRC_WIDTH;
21. stDst.u32Height = SRC_HEIGHT;
22. stDst.u32Stride = SRC_WIDTH * 2;
23. stDst.phyAddr = phyAddr;

```

```
24.
25. memset(&stLine, 0x0, sizeof(stLine));
26. stLine.stPointFrom.s16x = 0;
27. stLine.stPointFrom.s16y = 0;
28. stLine.stPointTo.s16x = 100;
29. stLine.stPointTo.s16y = 100;
30. stLine.u16Width = 1;
31. stLine.bColorGradient = TRUE;
32. stLine.bColorFrom = 0xffff0000;
33. stLine.bColorTo = 0xffff00ff;
34.
35. ExecFunc(MI_GFX_DrawLine(&stDst, &stLine, &u16TargetFence),
MI_SUCCESS);
36. ExecFunc(MI_GFX_WaitAllDone(FALSE, u16TargetFence),
MI_SUCCESS);
37. if (NULL != fp)
38. {
39.     fwrite(pVirAddr, 1, SRC_WIDTH*SRC_HEIGHT*2, fp);
40.     fclose(fp);
41.     fp = NULL;
42. }
43. ExecFunc(MI_SYS_Munmap(pVirAddr, SRC_WIDTH*SRC_HEIGHT*2),
MI_SUCCESS);
44. ExecFunc(MI_SYS_MMA_Free(phyAddr), MI_SUCCESS);
45.
46. ExecFunc(MI_GFX_Close(), MI_SUCCESS);
47. ExecFunc(MI_SYS_Exit(), MI_SUCCESS);
```

### 3. GFX data type

#### 3.1. GFX data type description

| type of data                              | illustrate                                    |
|-------------------------------------------|-----------------------------------------------|
| MI_GFX_ColorFmt_e [#32-mi_gfx_colorfmt_e] | Define the pixel color format                 |
| MI_GFX_Surface_t [#33-mi_gfx_surface_t]   | Define the bitmap property structure          |
| MI_GFX_Rect_t [#34-mi_gfx_rect_t]         | Define the operation area attribute structure |

|                                                                      |                                                             |
|----------------------------------------------------------------------|-------------------------------------------------------------|
| <a href="#">MI_GFX_ColorKeyOp_e</a> [#35-mi_gfx_colorkeyop_e]        | Define Colorkey operation mode                              |
| <a href="#">MI_GFX_ColorKeyValue_t</a> [#36-mi_gfx_colorkeyvalue_t]  | Define the Colorkey color property structure                |
| <a href="#">MI_GFX_ColorKeyInfo_t</a> [#37-mi_gfx_colorkeyinfo_t]    | Define the Colorkey property structure                      |
| <a href="#">MI_GFX_Rotate_e</a> [#38-mi_gfx_rotate_e]                | Define the image rotation angle                             |
| <a href="#">MI_GFX_Mirror_e</a> [#39-mi_gfx_mirror_e]                | Define image mirroring mode                                 |
| <a href="#">MI_GFX_DfbBldOp_e</a> [#310-mi_gfx_dfbldop_e]            | Define the mode of Alpha Blending                           |
| <a href="#">MI_Gfx_DfbBlendFlags_e</a> [#311-mi_gfx_dfbblendflags_e] | Flags that define Alpha Blending                            |
| <a href="#">MI_GFX_Opt_t</a> [#312-mi_gfx_opt_t]                     | Optional attribute structure defining the Bitblit operation |
| <a href="#">MI_GFX_PaletteEntry_t</a> [#313-mi_gfx_paletteentry_t]   | Define the palette color structure                          |
| <a href="#">MI_GFX_Palette_t</a> [#314-mi_gfx_palette_t]             | Define Palette Structure                                    |
| <a href="#">MI_GFX_InitParam_t</a> [#315-mi_gfx_initparam_t]         | Define GFX device initialization parameters                 |
| <a href="#">MI_GFX_Point_t</a> [#316-mi_gfx_point_t]                 | define point attribute structure                            |
| <a href="#">MI_GFX_Line_t</a> [#317-mi_gfx_line_t]                   | Define Line Properties Structure                            |

### 3.2. MI\_GFX\_ColorFmt\_e

- illustrate

Defines the pixel color format.

- definition

```
typedef enum
{
    E_MI_GFX_FMT_I1 = 0, /* ColorFormat */
    E_MI_GFX_FMT_I2,
    E_MI_GFX_FMT_I4 ,
    E_MI_GFX_FMT_I8,
    E_MI_GFX_FMT_FABAFGBG2266,
    E_MI_GFX_FMT_1ABFGBG12355,
    E_MI_GFX_FMT_RGB565,
    E_MI_GFX_FMT_ARGB1555,
    E_MI_GFX_FMT_ARGB4444,
    E_MI_GFX_FMT_ARGB1555_DST,
    E_MI_GFX_FMT_YUV422,
    E_MI_GFX_FMT_ARGB8888,
    E_MI_GFX_FMT_RGBA5551,
    E_MI_GFX_FMT_RGBA4444,
    E_MI_GFX_FMT_ABGR8888,
    E_MI_GFX_FMT_BGRA5551,
    E_MI_GFX_FMT_ABGR1555,
    E_MI_GFX_FMT_ABGR4444,
    E_MI_GFX_FMT_BGRA4444,
    E_MI_GFX_FMT_BGR565,
    E_MI_GFX_FMT_RGBA8888,
```

```

        E_MI_GFX_FMT_BGRA8888,

        E_MI_GFX_FMT_MAX

    } MI_GFX_ColorFmt_e;

```

- Precautions

Corresponding to the color format supported by the surface, the corresponding color format of SrcSurface and DstSurface during bitblit operation.

- Related data types and interfaces

[MI\\_GFX\\_ColorKeyInfo\\_t](#) [#37-mi\_gfx\_colorkeyinfo\_t]

[MI\\_GFX\\_Surface\\_t](#) [#33-mi\_gfx\_surface\_t]

[MI\\_GFX\\_SetPalette](#) [#29-mi\_gfx\_setpalette]

### 3.3. MI\_GFX\_Surface\_t

- illustrate

Define the bitmap property structure.

- definition

```

typedef struct MI_GFX_Surface_s
{
    MI_PHY phyAddr;

    MI_GFX_ColorFmt_e eColorFmt;

    MI_U32 u32Width ;

    MI_U32 u32Height;

    MI_U32 u32Stride;

} MI_GFX_Surface_t;

```

- member

| member     | describe                                                      |
|------------|---------------------------------------------------------------|
| u32PhyAddr | The bitmap corresponds to the physical address of the memory. |
| eColorFmt  | Bitmap pixel color format.                                    |
| u32Height  | Bitmap height.                                                |
| u32Width   | Bitmap width.                                                 |
| u32Stride  | Bitmap span.                                                  |

- Precautions
  - When the pixel format size of the bitmap is greater than or equal to 1Byte, the physical address of the bitmap corresponding to the memory and the Stride must be aligned according to the pixel format size; when the pixel format size of the bitmap is less than 1Byte, the physical address of the bitmap corresponding memory and the Stride need Aligned according to 1Byte.
  - The horizontal start position and width of a bitmap in YUYV422 format must be an even number.

- Related data types and interfaces

[MI\\_GFX\\_QuickFill](#) [#25-mi\_gfx\_quickfill]

[MI\\_GFX\\_DrawLine](#) [#212-mi\_gfx\_drawline]

[MI\\_GFX\\_Bitblit](#) [#28-mi\_gfx\_bitblit]

---

### 3.4. MI\_GFX\_Rect\_t

- illustrate

Define the operation area structure.
- definition

```
typedef struct MI_GFX_Rect_s
{
    MI_S32  s32Xpos;

    MI_S32  s32Ypos;

    MI_U32   u32Width ;

    MI_U32   u32Height;

} MI_GFX_Rect_t;
```

- member

| member    | describe                                                                                          |
|-----------|---------------------------------------------------------------------------------------------------|
| s32Xpos   | The starting abscissa of the operation area, in pixels. valid range [0, bitmap width)             |
| s32Ypos   | The starting vertical coordinate of the operation area, in pixels. valid range [0, bitmap height) |
| u32Width  | The width of the operation area, in pixels.                                                       |
| u32Height | The height of the operation area, in pixels.                                                      |

- Precautions

The operation area must be within the range of the bitmap.

- Related data types and interfaces

[MI\\_GFX\\_Opt\\_t](#) [#312-mi\_gfx\_opt\_t]

[MI\\_GFX\\_QuickFill](#) [#25-mi\_gfx\_quickfill]

[MI\\_GFX\\_Bitblit](#) [#28-mi\_gfx\_bitblit]

---

### 3.5. MI\_GFX\_ColorKeyOp\_e

- illustrate  
Defines the colorkey operation mode.
- definition

```
typedef enum
{
    E_MI_GFX_RGB_OP_EQUAL = 0,
    E_MI_GFX_RGB_OP_NOT_EQUAL,
    E_MI_GFX_ALPHA_OP_EQUAL,
    E_MI_GFX_ALPHA_OP_NOT_EQUAL,
    E_MI_GFX_ARGB_OP_EQUAL,
    E_MI_GFX_ARGB_OP_NOT_EQUAL,
    E_MI_GFX_CKEY_OP_BUTT,
} MI_GFX_ColorKey_e;
```

- member

| member                       | describe                                    |
|------------------------------|---------------------------------------------|
| E_MI_GFX_RGB_OP_EQUAL        | RGB is equal to do colorkey operation       |
| E_MI_GFX_RGB_OP_NOT_EQUAL    | RGB is not equal to do colorkey operation   |
| E_MI_GFX_ ALPHA_OP_EQUAL     | ALPHA is equal to do colorkey operation     |
| E_MI_GFX_ ALPHA_OP_NOT_EQUAL | ALPHA is not equal to do colorkey operation |
| E_MI_GFX_ ARGB_OP_EQUAL      | ARGB equal to do colorkey operation         |
| E_MI_GFX_ARGB_OP_NOT_EQUAL   | ARGB is not equal to do colorkey operation  |
| E_MI_GFX_CKEY_OP_BUTT        | Invalid colorkey operation                  |

- Related data types and interfaces

[MI\\_GFX\\_ColorKeyInfo\\_t](#) [#37-mi\_gfx\_colorkeyinfo\_t]

---

### 3.6. MI\_GFX\_ColorKeyValue\_t

- illustrate

Defines the color structure for colorkey.

- definition

```
typedef struct MI_GFX_ColorKey_s
{
    MI_U32 u32ColorStart;

    MI_U32 u32ColorEnd;

} MI_GFX_ColorKeyValue_t;
```

- member

| member        | describe                         |
|---------------|----------------------------------|
| u32ColorStart | ColorKey color range start value |
| u32ColorEnd   | ColorKey color range end value   |

- Precautions

When doing colorkey operation for only one color value, set u32ColorStart = u32ColorEnd = colorVal.

- Related data types and interfaces

[MI\\_GFX\\_ColorKeyInfo\\_t](#) [#37-mi\_gfx\_colorkeyinfo\_t]

---

### 3.7. MI\_GFX\_ColorKeyInfo\_t

- illustrate  
Define the colorkey property structure.
- definition

```
typedef struct MI_GFX_ColorKeyInfo_s
{
    MI_BOOL    bEnColorKey ;

    MI_GFX_ColorKeyOp_e  eKeyOp ;

    MI_GFX_ColorFmt_e  eKeyFmt;

    MI_GFX_ColorKeyValue_t  stCKeyVal;

} MI_GFX_ColorKeyInfo_t;
```

- member

| member      | describe                             |
|-------------|--------------------------------------|
| bEnColorKey | Whether to enable ColorKey operation |
| eKeyOp      | ColorKey Operation Mode              |
| stCKeyVal   | ColorKey color range                 |
| eKeyFmt     | ColorKey's color format              |

- Related data types and interfaces  
[MI\\_GFX\\_Opt\\_t](#) [#312-mi\_gfx\_opt\_t]

3.8. MI\_GFX\_Rotate\_e

- illustrate  
Defines the image rotation angle.

- definition

```
typedef enum
{
    E_MI_GFX_ROTATE_0 = 0,
    E_MI_GFX_ROTATE_90,
    E_MI_GFX_ROTATE_180,
    E_MI_GFX_ROTATE_270
} MI_GFX_Rotate_e;
```

- Related data types and interfaces

[MI\\_GFX\\_Opt\\_t](#) [#312-mi\_gfx\_opt\_t]

---

### 3.9. MI\_GFX\_Mirror\_e

- illustrate

Defines the image mirroring mode.

- definition

```
typedef enum
{
    E_MI_GFX_MIRROR_NONE = 0,
    E_MI_GFX_MIRROR_HORIZONTAL,
    E_MI_GFX_MIRROR_VERTICAL,
    E_MI_GFX_MIRROR_BOTH,
    E_MI_GFX_MIRROR_MAX
} MI_GFX_Mirror_e;
```

- member

| member                     | describe                                     |
|----------------------------|----------------------------------------------|
| E_MI_GFX_MIRROR_NONE       | The output image is not mirrored             |
| E_MI_GFX_MIRROR_HORIZONTAL | Mirror the output image horizontally         |
| E_MI_GFX_MIRROR_VERTICAL   | The output image is mirrored vertically      |
| E_MI_GFX_MIRROR_BOTH       | Output image horizontal + vertical mirroring |
| E_MI_GFX_MIRROR_MAX        | Invalid image type                           |

- Related data types and interfaces

[MI\\_GFX\\_Opt\\_t](#) [#312-mi\_gfx\_opt\_t]

---

### 3.10. MI\_GFX\_DfbBldOp\_e

- illustrate

Defines the mode for Alpha Blending.

- definition

```
typedef enum
{
    E_MI_GFX_DFB_BLD_ZERO = 0,
    E_MI_GFX_DFB_BLD_ONE,
    E_MI_GFX_DFB_BLD_SRCCOLOR,
    E_MI_GFX_DFB_BLD_INVSRCOLOR,
    E_MI_GFX_DFB_BLD_SRCALPHA,
    E_MI_GFX_DFB_BLD_INVSRCALPHA,
```

```
E_MI_GFX_DFB_BLD_DESTALPHA,  
  
E_MI_GFX_DFB_BLD_INVDESTALPHA,  
  
E_MI_GFX_DFB_BLD_DESTCOLOR,  
  
E_MI_GFX_DFB_BLD_INVDESTCOLOR,  
  
E_MI_GFX_DFB_BLD_SRCALPHASAT,  
  
E_MI_GFX_DFB_BLD_NONE,  
  
E_MI_GFX_DFB_BLD_MAX,  
  
} MI_GFX_DfbBldOp_e;
```

- member

| member                           | describe                                               |
|----------------------------------|--------------------------------------------------------|
| E_MI_GFX_DFB_BLD_OP_ZERO         | argb *= 0.0<br>同directfb DSBF_ZERO                     |
| E_MI_GFX_DFB_BLD_OP_ONE          | argb *= 1.0<br>same as directfb DSBF_ONE               |
| E_MI_GFX_DFB_BLD_OP_SRCOLOR      | argb *= Sargb<br>same as directfb DSBF_SRCOLOR         |
| E_MI_GFX_DFB_BLD_OP_INVSRCOLOR   | argb *= 1.0 - Sargb<br>同directfb DSBF_INVSRCOLOR       |
| E_MI_GFX_DFB_BLD_OP_SRCALPHA     | argb *= Saaaa<br>同directfb DSBF_SRCALPHA               |
| E_MI_GFX_DFB_BLD_OP_INVSRCALPHA  | argb *= 1.0 - Saaaa<br>同directfb DSBF_INVSRCALPHA      |
| E_MI_GFX_DFB_BLD_OP_DESTCOLOR    | argb *= Dargb<br>同directfb DSBF_DESTCOLOR              |
| E_MI_GFX_DFB_BLD_OP_INVDESTCOLOR | argb *= 1.0 - Dargb<br>同directfb DSBF_INVDESTCOLOR     |
| E_MI_GFX_DFB_BLD_OP_DESTALPHA    | argb *= Data<br>同directfb DSBF_DESTALPHA               |
| E_MI_GFX_DFB_BLD_OP_INVDESTALPHA | argb *= 1.0 - Daaaa<br>同directfb DSBF_INVDESTALPHA     |
| E_MI_GFX_DFB_BLD_OP_SRCALPHASAT  | rgb *= min(Sa, 1-Da)<br>same directfb DSBF_SRCALPHASAT |
| E_MI_GFX_BLD_MAX                 | Invalid alpha blending mode                            |

- Precautions

When superimposing the source bitmap and the target bitmap, you can set the superposition mode of the Src channel and the Dst channel respectively.

Now supports 11 overlay modes, use [MI\\_GFX\\_DfbBldOp\\_e](#) [#310-mi\_gfx\_dfbldop\_e] to set the mode.

- Related data types and interfaces

[MI\\_GFX\\_Opt\\_t](#) [#312-mi\_gfx\_opt\_t]

---

### 3.11. MI\_Gfx\_DfbBlendFlags\_e

- illustrate

Flags that define Alpha Blending

- definition

```
typedef enum
{
    E_MI_GFX_DFB_BLEND_NOFX = 0x00000000,
    E_MI_GFX_DFB_BLEND_COLORALPHA = 0x00000001,
    E_MI_GFX_DFB_BLEND_ALPHACHANNEL = 0x00000002,
    E_MI_GFX_DFB_BLEND_COLORIZE = 0x00000004,
    E_MI_GFX_DFB_BLEND_SRC_PREMULTIPLY = 0x00000008,
    E_MI_GFX_DFB_BLEND_SRC_PREMULTCOLOR = 0x00000010,
    E_MI_GFX_DFB_BLEND_DST_PREMULTIPLY = 0x00000020,
    E_MI_GFX_DFB_BLEND_XOR = 0x00000040,
    E_MI_GFX_DFB_BLEND_DEMULTIPLY = 0x00000080,
    E_MI_GFX_DFB_BLEND_SRC_COLORKEY = 0x00000100,
    E_MI_GFX_DFB_BLEND_DST_COLORKEY = 0x00000200,
    E_MI_GFX_DFB_BLEND_MAX = 0x3FF
} MI_Gfx_DfbBlendFlags_e;
```

- member

| member                               | describe                                                                                                                                                         |
|--------------------------------------|------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| E_MI_GFX_DFB_BLEND_NOFX              | without any BLD operation                                                                                                                                        |
| E_MI_GFX_DFB_BLEND_COLORALPHA        | src alpha will operate according to<br>const color value $\text{src.a} = \text{const color.a}$ ;<br>same as directfb<br>DSBLIT_BLEND_COLORALPHA                  |
| E_MI_GFX_DFB_BLEND_ALPHACHANNEL      | src alpha will operate according to<br>const color value $\text{src.a} = \text{src.a} * \text{const color.a}$ ;<br>same as directfb<br>DSBLIT_BLEND_ALPHACHANNEL |
| E_MI_GFX_DFB_BLEND_COLORIZE          | Use const color as the global color<br>and adjust the source color<br>same as directfb DSBLIT_COLORIZE                                                           |
| E_MI_GFX_DFB_BLEND_SRC_PREMULTIPLY   | Premultiply source color with source<br>alpha $\text{src.r} = \text{src.r} * \text{src.a}$ ;<br>same as directfb<br>DSBLIT_SRC_PREMULTIPLY                       |
| E_MI_GFX_DFB_BLEND_SRC_PREMULTICOLOR | Premultiply source color with const<br>alpha $\text{src.r} = \text{src.r} * \text{const.a}$ ;<br>same as directfb<br>DSBLIT_SRC_PREMULTICOLOR                    |
| E_MI_GFX_DFB_BLEND_DST_PREMULTIPLY   | Premultiply destination color with<br>destination alpha $\text{dst.r} = \text{dst.r} * \text{dst.a}$ ;<br>same as directfb<br>DSBLIT_DST_PREMULTIPLY             |
| E_MI_GFX_DFB_BLEND_XOR               | Adjust and premultiply alpha $\text{src.a} \wedge = \text{dst.a}$ ;<br>same as directfb DSBLIT_XOR                                                               |
| E_MI_GFX_DFB_BLEND_DEMULTIPLY        | Premultiplied to non-premultiplied<br>conversion, it is better not to use such                                                                                   |

|                                 |                                                                                             |
|---------------------------------|---------------------------------------------------------------------------------------------|
|                                 | operations $xr = ((int)xr << 8) / ((int)xa + 1)$ ; Equivalent to directfb DSBLIT_DEMULTIPLY |
| E_MI_GFX_DFB_BLEND_SRC_COLORKEY | If the color of src is equal to const color, it will be keyed                               |
| E_MI_GFX_DFB_BLEND_DST_COLORKEY | If the color of dst is equal to const color, it will be keyed                               |
| E_MI_GFX_DFB_BLEND_MAX          | All the above actions will be enabled                                                       |

- Related data types and interfaces

[MI\\_GFX\\_Opt\\_t](#) [#312-mi\_gfx\_opt\_t]

### 3.12. MI\_GFX\_Opt\_t

- illustrate  
Optional attribute structure defining the Bitblit operation
- definition

```
typedef struct MI_GFX_Opt_s
{
    MI_GFX_Rect_t stClipRect;

    MI_GFX_ColorKeyInfo_t stSrcColorKeyInfo;

    MI_GFX_ColorKeyInfo_t stDstColorKeyInfo;

    MI_GFX_DfbBldOp_e eSrcDfbBldOp;

    MI_GFX_DfbBldOp_e eDstDfbBldOp;

    MI_GFX_Mirror_e eMirror;

    MI_GFX_Rotate_e eRotate;

    MI_Gfx_DfbBlendFlags_e eDFBBlendFlag;
```

```
MI_U32 u32GlobalSrcConstColor;  
  
MI_U32 u32GlobalDstConstColor;  
  
} MI_GFX_Opt_t;
```

• member

| member                 | describe                                                                                                                         |
|------------------------|----------------------------------------------------------------------------------------------------------------------------------|
| stSrcColorKeyInfo      | Src surface colorkey operation                                                                                                   |
| stDstColorKeyInfo      | Dst surface color key operation                                                                                                  |
| stClipRect             | clip area                                                                                                                        |
| eMirror                | Image mirror type                                                                                                                |
| eSrcDfbBldOp           | Modes of Src Surface BLEND Operation                                                                                             |
| eDstDfbBldOp           | Modes of Dst Surface BLEND Operation                                                                                             |
| eRotate                | Image rotation angle, 0, 90, 180, 270 degree rotation                                                                            |
| eDFBBlendFlag          | Alpha Blending logo, such as XOR, AlphaBlending, Colorize and other operations                                                   |
| u32GlobalSrcConstColor | For the const parameters corresponding to the eDFBBlendFlag operation, the eDFBBlendFlag operation has corresponding mutli, etc. |
| u32GlobalDstConstColor | For the const parameters corresponding to the eDFBBlendFlag operation, the eDFBBlendFlag operation has corresponding mutli, etc. |

• Precautions

Currently only u32GlobalSrcConstColor is valid.

• Related data types and interfaces

## MI\_GFX\_Bitblit [#28-mi\_gfx\_bitblit]

---

### 3.13. MI\_GFX\_PaletteEntry\_t

- illustrate

Defines the palette color structure.

- definition

```
typedef union
{
    ///  ARGB8888  byte  order

    struct
    {
        MI_U8  u8A  ;

        MI_U8  u8R  ;

        MI_U8  u8G  ;

        MI_U8  u8B  ;

    } RGB;

    // u8Data[0] = u8A

    //  u8Data [ 1 ]  =  u8R

    //  u8Data [ 2 ]  =  u8G

    //  u8Data [ 3 ]  =  u8B

    MI_U8  u8Data [ 4 ];

} MI_GFX_PaletteEntry_t;
```

- member

| member  | describe                                                                                 |
|---------|------------------------------------------------------------------------------------------|
| RGB     | The component structure representation of the RGB color corresponding to the index color |
| u8 Date | The byte order representation of the RGB color corresponding to the indexed color        |

- Precautions  
Pay attention to the description of the color definition in the comments in the definition so that the palette data can be filled in correctly.

- Relevant data types and results  
[MI\\_GFX\\_Palette\\_t](#) [#314-mi\_gfx\_palette\_t]

---

### 3.14. MI\_GFX\_Palette\_t

- illustrate  
Defines the palette structure.
- definition

```
typedef struct MI_GFX_Palette_s
{
    MI_GFX_PaletteEntry_t aunPalette[256];

    MI_U16  u16PalStart ;

    MI_U16  u16PalEnd ;

}MI_GFX_Palette_t;
```

- member

| member      | describe                                                            |
|-------------|---------------------------------------------------------------------|
| aunPalette  | RGB color array corresponding to palette index                      |
| u16PalStart | Need to configure the starting index of the index set of RGB colors |
| u16PalEnd   | Need to configure the index set end index of RGB color              |

- Relevant data types and results  
[MI\\_GFX\\_PaletteEntry\\_t](#) [#313-mi\_gfx\_paletteentry\_t]  
[MI\\_GFX\\_SetPalette](#) [#29-mi\_gfx\_setpalette]

3.15. MI\_GFX\_InitParam\_t

- illustrate  
Defines GFX device initialization parameters.
- definition

```
typedef struct MI_GFX_InitParam_s
{
    MI_U32  u32DevId;

    MI_U8   * u8Data ;
} MI_GFX_InitParam_t;
```

- member

| member name | describe            |
|-------------|---------------------|
| u32DevId    | Device ID           |
| u8 Date     | data pointer buffer |

- Related data types and interfaces  
[MI\\_GFX\\_InitDev](#) [#210-mi\_gfx\_initdev]
- 

### 3.16. MI\_GFX\_Point\_t

- illustrate  
Define the point attribute structure.
- definition

```
typedef struct MI_GFX_Point_s
{
    MI_S16 s16x;
    MI_S16 s16y;
} MI_GFX_Point_t;
```

- member

| member name | describe     |
|-------------|--------------|
| s16x        | X coordinate |
| S16y        | Y coordinate |

- Related data types and interfaces  
[MI\\_GFX\\_Line\\_t](#) [#317-mi\_gfx\_line\_t]
- 

### 3.17. MI\_GFX\_Line\_t

- illustrate  
Define the line property structure.

- definition

```
typedef struct MI_GFX_Line_s
{
    MI_GFX_Point_t stPointFrom;

    MI_GFX_Point_t srPointTo;

    MI_U16  u16Width ;

    MI_BOOL bColorGradient;

    MI_U32  u32ColorFrom;

    MI_U32  u32ColorTo;

} MI_GFX_Line_t;
```

- member

| member name    | describe                   |
|----------------|----------------------------|
| stPointFrom    | starting point of the line |
| srPointTo      | end point of the line      |
| u16Width       | Line width                 |
| bColorGradient | Whether the color gradient |
| u32ColorFrom   | Gradient start color       |
| u32ColorTo     | gradient end color         |

- Related data types and interfaces

[MI\\_GFX\\_DrawLine](#) [#212-mi\_gfx\_drawline]

.....

## 4. Error code

GFX API error codes are as follows

| error code | macro definition             | describe                       |
|------------|------------------------------|--------------------------------|
| 0xA00D2003 | MI_ERR_GFX_INVALID_PARAM     | illegal parameter              |
| 0xA00D2002 | MI_ERR_GFX_DEV_BUSY          | GFX device busy                |
| 0xA00D2220 | MI_ERR_GFX_NOT_INIT          | Device not initialized         |
| 0xA00D2221 | MI_ERR_GFX_DRV_NOT_SUPPORT   | Unsupported operation type     |
| 0xA00D2222 | MI_ERR_GFX_DRV_FAIL_FORMAT   | unsupported format type        |
| 0xA00D2223 | MI_ERR_GFX_NON_ALIGN_ADDRESS | address is not aligned         |
| 0xA00D2224 | MI_ERR_GFX_NON_ALIGN_PITCH   | Pitch alignment error          |
| 0xA00D2225 | MI_ERR_GFX_DRV_FAIL_OVERLAP  | Option memory has overlay area |
| 0xA00D2226 | MI_ERR_GFX_DRV_FAIL_STRETCH  | Zoom failed                    |
| 0xA00D2228 | MI_ERR_GFX_DRV_FAIL_LOCKED   | Device locked by another user  |
| 0xA00D2229 | MI_ERR_GFX_DRV_FAIL_BLTADDR  | Bitblit start address error    |

---

## 5. Introduction to PROCFS

### 5.1. cat

- debug info

```
# cat /proc/mi_modules/mi_gfx/mi_gfx0
```

```
-----Gfx Dev Attr-----
module init count:  2
Device id:  0
ARGB1555 alpha threshold:  1
ARGB1555 To ARGB8888 alpha value:  255
Scaling coefficient:  500 / 1000

Device id:  1
ARGB1555 alpha threshold:  1
ARGB1555 To ARGB8888 alpha value:  255
Scaling coefficient:  500 / 1000
-----
```

- Analysis of debug information

Recording the current usage of GFX and device properties can dynamically obtain this information, which is convenient for debugging and testing.

- Parameter Description

| parameter   |                                  | describe                                                                                                                                                                                                                                                                                                                       |
|-------------|----------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| device info | module init count                | The number of times the device is opened, because GFX will have multiple modules to call, and it is not sure which module will call the initialization of GFX first, so the device allows multiple Opens, and the number of Opens will be recorded to count when it is Closed. Determine whether you need a real Close Device. |
|             | Device id                        | Device ID for GFX                                                                                                                                                                                                                                                                                                              |
|             | ARGB1555 alpha threshold         | When ARGB8888 is converted to ARGB1555, the alpha determination threshold inside GFX, if it is greater than N, alpha is 1, otherwise, alpha is 0.                                                                                                                                                                              |
|             | ARGB1555 To ARGB8888 alpha value | When converting ARGB1555 to ARGB8888, GFX converts the alpha of the pixel whose ARGB1555 alpha bit is 1 to ARGB1555 To ARGB8888 alpha value.                                                                                                                                                                                   |
|             | Scaling coefficient              | Image geometric center offset                                                                                                                                                                                                                                                                                                  |